Student notes — Session 11 · Characters and strings

Slides: [`Class_11_Characters_and_strings.pdf`](../slides/Class_11_Characters_and_strings.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Characters and strings

The programs of this class are on Blackboard: Week 6 module, *Class 11 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Until now every value in your programs was a number. Today your programs start to work with text: one character, and then whole words and lines.

Slide 2 — Quiz 3 is this Friday

Quiz 3 is on **Friday 16 October**, in the first 25 minutes of class. It covers classes 7 to 10: functions, scope, call-by-value, recursion, arrays, search, min and max, bubble sort. Nothing from today. Paper and pen, no laptop. Knowledge checks 7 to 10 and the practice tests on Blackboard are the best preparation.

Slide 3 — What Friday gave us

- You send an array to a function together with its size: `f(a, n)`.
- A function can change the caller's array.
- A search returns the index, or `-1` after the loop.
- Min and max start from `a[0]`.

Slide 4 — Recap: where does it stop?

Try it before you look at the next slide: what are the two lines? There is no `i < 6` in the loop. What stops it?

Slide 5 — Recap: where does it stop?

The loop runs while `a[i] != 0`. At `i = 5` it meets the 0 and stops. It prints `4 8 15 16 23 ` and then `count: 5`.

The 0 is an **end mark**: the loop does not need the size of the array. Strings work in exactly this way. File: `01_recap_stop_at_zero.c`.

Slide 6 — Today's roadmap

One character, a character is a number, testing characters, strings, reading strings, loops over a string.

Slide 7 — One character

The type `char`.

Slide 8 — A char holds one character

A `char` holds **one** character: a letter, a digit, a space or a sign. Write it in **single** quotes and print it with `%c`:

```
char grade = 'A';
printf("%c\n", grade);     // prints A
```

Slide 9 — Three characters

```
char grade = 'A';
char digit = '7';
char space = ' ';
printf("[%c] [%c] [%c]\n", grade, digit, space);
```

It prints `[A] [7] [ ]`. The brackets are only in the `printf`, so you can see the space. `'7'` is the **character** 7, the symbol on the keyboard key, not the number 7. File: `02_char_basics.c`.

Slide 10 — A char is a small number

Inside the computer every character is stored as a whole number, its **code**. `'A'` is 65, `'a'` is 97, `'0'` is 48. This table of codes is called ASCII. The same value prints as a character with `%c` and as a number with `%d`.

Slide 11 — The same value, two ways

`printf("%c = %d\n", 'A', 'A');` prints `A = 65`. The program prints:

```
A = 65
B = 66
a = 97
0 = 48
9 = 57
[ ] = 32
```

File: `03_ascii.c`.

Slide 12 — Codes to remember

- `'A'` to `'Z'` are 65 to 90, in order.
- `'a'` to `'z'` are 97 to 122, in order.
- `'0'` to `'9'` are 48 to 57, in order.
- The space is 32.

You do not need to learn the numbers. Remember the **order**: it is what makes arithmetic on characters work.

Slide 13 — A loop over the alphabet

```
for (char c = 'A'; c <= 'Z'; c++) {
    printf("%c", c);
}
```

`c++` adds 1 to the code, so `'A'` (65) becomes `'B'` (66). It prints `ABCDEFGHIJKLMNOPQRSTUVWXYZ`. File: `04_alphabet.c`.

Slide 14 — A character is a number

Because a character is a number, you can calculate with it.

Slide 15 — Predict: letters plus numbers

Try it before you look at the next slide: `'a'` is 97. What are the four lines?

Slide 16 — Letters plus numbers

- `c + 1` is 98. Printed with `%c` it is `b`; printed with `%d` it is `98`.
- `'A' + 25` is 90, the code of `Z`.
- `x - 'a' + 1` with `x = 'e'` is 5.

The program prints `b`, `98`, `Z`, `e: letter 5`. File: `05_char_arith.c`.

Slide 17 — The place of a letter

`letter - 'a'` is the place of the letter, counting from 0:

- `'a' - 'a'` is 0
- `'b' - 'a'` is 1
- `'e' - 'a'` is 101 - 97 = 4

Add 1 to count from 1: e is letter 5.

Slide 18 — A small letter becomes a capital

```
char c = 'g';
char big = c - 'a' + 'A';
```

`c - 'a'` is 6, the place of g. Adding `'A'` gives the capital at place 6: `'G'`. It prints `g becomes G` and `103 becomes 71`. A small letter and its capital are always 32 apart. File: `06_case_by_hand.c`.

Slide 19 — Predict: the character 7

Try it before you look at the next slide: `d` holds `'7'`. What are the three numbers?

Slide 20 — The character 7 is not the number 7

```
char d = '7';
printf("%d\n", d);              // 55
printf("%d\n", d - '0');        // 7
printf("%d\n", (d - '0') * 2);  // 14
```

The character `'7'` has the code 55. Subtract `'0'` (48) and you get 7, the value of the digit. File: `07_digit_value.c`.

Slide 21 — A digit and its value

The value of a digit character `c` is `c - '0'`. `'0' - '0'` is 0, `'7' - '0'` is 7, `'9' - '0'` is 9. It works because the digits have codes in order. Class 12 uses this to turn a whole string like `"2026"` into a number.

Slide 22 — Careful: which quotes?

Try it before you look at the next slide: `char c = "A";` with double quotes. Does it compile?

Slide 23 — Double quotes: gcc refuses

On the course compiler (gcc 16) it does not compile:

```
error: initialization of 'char' from 'char *' makes integer from pointer without a cast
```

`"A"` in double quotes is a **string**, not one character. A `char` takes `'A'` in single quotes. (An older gcc only warns and prints a strange character. It is wrong either way.) File: `08_bug_char_quotes.c`.

Slide 24 — 'A' or "A"?

- `'A'`: one character, the number 65. It goes in a `char`.
- `"A"`: a string. It takes **two** boxes: `A` and the end mark `'\0'`. It goes in a `char` array.

Slide 25 — Testing characters

Is it a letter? A digit? A capital?

Slide 26 — Which kind of character?

Because letters and digits are in order, a range test works:

```
if (c >= 'a' && c <= 'z') {
    printf("%c is a small letter\n", c);
} else if (c >= 'A' && c <= 'Z') {
    printf("%c is a capital letter\n", c);
} else if (c >= '0' && c <= '9') {
    printf("%c is a digit\n", c);
}
```

With input `Q` it prints `Q is a capital letter`. The space in `scanf(" %c", &c)` skips spaces and Enter keys. File: `09_read_char.c`.

Slide 27 — <ctype.h> does it for you

Put `#include <ctype.h>` at the top of the program. Then:

- `isdigit(c)`, `isalpha(c)`, `isupper(c)`, `islower(c)` answer yes or no.
- `toupper(c)` and `tolower(c)` give the letter in the other case.
- If `c` is not a letter, `toupper` and `tolower` give it back unchanged.

Slide 28 — The six functions

call	question or job	example
`isdigit(c)`	a digit?	`'7'`: yes
`isalpha(c)`	a letter?	`'?'`: no
`isupper(c)`	a capital?	`'Q'`: yes
`islower(c)`	a small letter?	`'Q'`: no
`toupper(c)`	make it a capital	`'g'` gives `'G'`
`tolower(c)`	make it small	`'Q'` gives `'q'`

Slide 29 — ctype.h at work

```
printf("%c %c\n", toupper('g'), tolower('Q'));   // G q
printf("%c %c\n", toupper('G'), toupper('7'));   // G 7
if (isdigit('7')) {
    printf("7 is a digit\n");
```

```
}
if (!isalpha('?')) {
    printf("? is not a letter\n");
}
```

The `is...` functions are made for an `if`. `!` means "not". File: `10_ctype.c`.

Slide 30 — Careful: what is "yes"?

Try it before you look at the next slide: what are the three numbers?

Slide 31 — "Yes" is not always 1

On the course compiler it prints `1`, `2`, `0`. `isalpha('x')` gives 2.

In C, "no" is 0 and "yes" is **any number that is not 0**. So:

```
if (isalpha(c))         // right
if (isalpha(c) == 1)    // wrong: false when isalpha gives 2
```

File: `11_ctype_true.c`.

Slide 32 — The same test, with ctype.h

The same program, shorter, with `isupper`, `islower`, `isdigit`. For a small letter it also prints `toupper(c)`; for a digit it prints `c - '0'`. With input `g` it prints `small, capital G`; with `4` it prints `digit, value 4`. File: `12_classify.c`.

Slide 33 — Strings

Text as an array of `char`.

Slide 34 — A string is an array of char

A **string** is an array of `char`: one character in each box, and after the last character one more box with the **end mark** `'\0'`, the null character. Its code is 0. A string in double quotes gets the `'\0'` automatically. It is like the 0 at the end of the recap array on slide 4.

Slide 35 — "hello" in memory

index	0	1	2	3	4	5
box	h	e	l	l	o	\0

Five letters, **six** boxes.

Slide 36 — Two ways to make a string

```
char w[6] = {'h', 'e', 'l', 'l', 'o', '\0'};   // box by box
char s[] = "hello";                              // the short way
```

With empty brackets gcc counts the boxes (6) and adds the `'\0'`. `printf("%s\n", s);` prints characters until it meets `'\0'`. Both lines print `hello`. File: `13_string_boxes.c`.

Slide 37 — Predict: how many boxes?

Try it before you look at the next slide: what are the two lines? What is in `s[2]`?

Slide 38 — "hi" takes three boxes

```
char s[] = "hi";
printf("%d %d %d\n", s[0], s[1], s[2]);
```

It prints `104 105 0`: the codes of h and i, then the `'\0'`, whose code is 0. File: `14_null_char.c`.

Slide 39 — Count the end mark

A string of n letters needs **n + 1** boxes. `"hi"` needs 3, `"hello"` needs 6. `char name[20];` has room for 19 letters and the `'\0'`.

Slide 40 — Careful: three boxes for "cat"

Try it before you look at the next slide: `char word[3] = "cat";` has no room for the `'\0'`. What is printed?

Slide 41 — No end mark: %s goes on

```
char next[] = "dog";
char word[3] = "cat";
printf("%s\n", word);
```

On the course compiler it prints `catdog`. With no `'\0'` after `t`, `%s` does not know where to stop: it goes on into the memory after the array, which here holds `dog`. On another computer it can print something else.

`gcc -Wall` gives **no warning** here. `gcc -Wall -Wextra` warns: `initializer-string for array of 'char' truncates NUL terminator`. The fix: `char word[4]` or `char word[]`. File: `15_bug_no_room.c`.

Slide 42 — Printing a string

```
char name[] = "Leyla";
printf("Hello, %s!\n", name);            // Hello, Leyla!
printf("First letter: %c\n", name[0]);   // First letter: L
```

`%s` with the array name prints the whole string. `%c` with one box prints one character. File: `16_print_string.c`.

Slide 43 — Change one box

A string is an array, so you can change a box:

```
char word[] = "cat";
word[0] = 'b';     // bat
word[2] = 'r';     // bar
```

It prints `cat`, `bat`, `bar`. File: `17_change_letter.c`.

Slide 44 — Careful: no = for a whole string

```
char word[] = "cat";    // fine: = when the array is made
word = "dog";           // error
```

gcc says `error: assignment to expression with array type`, the same error as `b = a;` for arrays in class 9. Class 12 shows the library function that copies a string. File: `18_bug_assign_string.c`.

Slide 45 — Reading strings

`scanf` and `fgets`.

Slide 46 — Read a word with scanf

```
char name[20];
printf("Your name: ");
scanf("%s", name);
printf("Hello, %s!\n", name);
```

With input `Leyla` it prints `Hello, Leyla!`. There is **no `&`** before `name`. File: `19_scanf_string.c`.

Slide 47 — Why no &?

- `scanf("%d", &n);` an `int` needs the `&`.
- `scanf("%s", name);` an array does not: its name already says where it is.

Why exactly? Pointers explain it, after the midterm.

Slide 48 — Predict: a full name

Try it before you look at the next slide: you type `Leyla Aliyeva`. What is printed?

Slide 49 — scanf stops at a space

It prints `Hello, Leyla!`. `%s` reads **one word**: it stops at a space or Enter. `Aliyeva` is not read into `name`.

Slide 50 — Careful: the & gets a warning

`scanf("%s", &name);` gets a warning from `gcc -Wall`:

```
warning: format '%s' expects argument of type 'char *', but argument 2 has type 'char
(*)[20]'
```

Argument 2 is `&name`. Delete the `&`. File: `20_bug_scanf_amp.c`.

Slide 51 — Read a whole line: fgets

```
char line[50];
fgets(line, 50, stdin);
printf("<%s>\n", line);
```

`fgets(where, how many boxes, stdin)` reads a whole line, spaces too. `stdin` is the keyboard. With input `Leyla Aliyeva` it prints:

```
<Leyla Aliyeva
>
```

The `>` is on the next line. File: `21_fgets.c`.

Slide 52 — fgets keeps the Enter key

`Leyla Aliyeva` is 13 characters (the space counts): boxes 0 to 12. `fgets` also stores the Enter key, `'\n'`, in box 13, and then `'\0'` in box 14. That `'\n'` is why the `>` went to the next line.

Slide 53 — scanf or fgets?

- `scanf("%s", s)`: one word, stops at a space, no `'\n'` stored.
- `fgets(s, 50, stdin)`: a whole line, spaces too, keeps the `'\n'`.

Slide 54 — Loops over a string

Walk along the string until the end mark.

Slide 55 — The string loop

```
for (int i = 0; s[i] != '\0'; i++) {
    // work with s[i]
}
```

No size is needed: the `'\0'` tells the loop where to stop. It is the recap loop of slide 4, with `'\0'` instead of 0.

Slide 56 — The length of "cat"

```
char s[] = "cat";
int len = 0;
while (s[len] != '\0') {
    len++;
}
```

`len` starts at 0, the first box. `s[0]` is `'c'`, not the end mark, so `len` becomes 1.

Slide 57 — The length of "cat"

`len`	`s[len]`	not `'\0'`?
0	`'c'`	yes
1	`'a'`	yes
2	`'t'`	yes
3	`'\0'`	no: stop

`len` is 3. It prints `length of cat: 3`. File: `22_length_loop.c`.

Slide 58 — Length as a function

```
int length(char s[]) {
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}
```

There is no size parameter. `length("Baku")` is 4, `length("Programming")` is 11, `length("")` is 0: the empty string holds only the `'\0'`. File: `23_length_function.c`.

Slide 59 — An int array and a string

- An `int` array does not know its size: `print_array(a, 5)`.
- A string carries its own end, the `'\0'`: `length(city)`.

Slide 60 — Remove the Enter key

```
for (int i = 0; line[i] != '\0'; i++) {
    if (line[i] == '\n') {
        line[i] = '\0';
    }
}
```

After `fgets` the line `Leyla Aliyeva` has length 14 (13 characters and the `'\n'`). The loop puts `'\0'` where the `'\n'` was. Now the length is 13 and it prints `<Leyla Aliyeva>` on one line. File: `24_remove_newline.c`.

Slide 61 — Predict: count the vowels

Try it before you look at the next slide: how many vowels does `count_vowels("Azerbaijan")` count?

Slide 62 — Counting vowels

It prints `vowels in Azerbaijan: 5`: A, e, a, i, a. The capital A counts because `tolower(s[i])` turns it into `'a'` first. File: `25_count_vowels.c`.

Slide 63 — Letters, digits, spaces

The program reads a line with `fgets` and counts with `isalpha`, `isdigit` and `line[i] == ' '`. For the input `Room B102, 16:00` it prints `letters 5, digits 7, spaces 2`. The comma and the `:` are none of the three. File: `26_count_kinds.c`.

Slide 64 — Predict: to_upper

Try it before you look at the next slide: `to_upper` returns nothing. What does `main` print?

Slide 65 — to_upper changes the string

```
void to_upper(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = toupper(s[i]);
    }
}
```

`main` prints `HELLO, WORLD 2026`. The function changes the caller's string, because an array is not copied (class 10). `toupper` leaves the comma, the space and the digits alone. File: `27_to_upper.c`.

Slide 66 — Class 12: string.h

Next class: the library <string.h> measures, copies, joins and compares strings (strlen, strcpy, strcat, strcmp). Today you wrote by hand the loop that strlen hides. Class 12 also turns "2026" into the number 2026 and back.

Slide 67 — Your turn

Two small string programs.

Slide 68 — Your turn: count_char

Write int count_char(char s[], char c). It returns how many times c is in s. count_char("banana", 'a') is 3. Try it before you look at the next slide.

Slide 69 — count_char: one answer

```
int count_char(char s[], char c) {
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == c) {
            count++;
        }
    }
    return count;
}
```

It prints 3 for 'a' and 0 for 'z'. File: 28_task_count_char.c.

Slide 70 — Your turn: backwards

Print a string backwards: "stressed" gives desserts. Hint: find the length first. Try it before you look at the next slide.

Slide 71 — Backwards: one answer

First count the length. Then print from len - 1 down to 0:

```
for (int i = len - 1; i >= 0; i--) {
    printf("%c", s[i]);
}
```

Start at len - 1, because box len holds the '\0'. It prints desserts. File: 29_task_backwards.c.

Slide 72 — Today in one slide

- A char is one character in single quotes, stored as a code.
- c - 'a' is the place of a letter; c - '0' is the value of a digit.
- <ctype.h>: isdigit, isalpha, isupper, islower, toupper, tolower. "Yes" is any number that is not 0.
- A string is char boxes ending in '\0', printed with %s.
- scanf("%s", s) reads one word, no &; fgets reads a line and keeps the '\n'.
- The string loop stops at '\0': no size needed.

Slide 73 — Before Friday

- **Knowledge check 11** on Blackboard, on this class.
- **Quiz 3** is on **Friday 16 October** and covers classes 7 to 10.
- This class's code: Blackboard, Week 6 module, *Class 11 code*.

Next class: `<string.h>`, and turning numbers into strings and back.